Uso de SDD + Antigravity orientado a servicios en el Laboratorio Virtual Auditivo

1. Objetivo del documento

Este documento explica cómo usar **SDD (Spec-Driven Development)** junto con **Antigravity** dentro del proyecto **Laboratorio Virtual Auditivo**, tomando como unidad de trabajo cada **servicio** del sistema.

El objetivo no es explicar el sistema completo ni repetir la documentación del proyecto, sino definir una forma clara y reutilizable para que el equipo pueda:

- especificar correctamente un servicio antes de implementarlo;
- organizar sus requerimientos en una carpeta;
- usar Antigravity de forma eficiente;
- revisar lo generado por IA antes de integrarlo al proyecto;
- mantener trazabilidad entre lo pedido, lo planificado y lo implementado.

El sistema ya cuenta con una idea funcional base: una plataforma donde profesionales de la audición pueden cargar datos, ejecutar simulaciones auditivas basadas en el modelo de Verhulst y visualizar resultados gráficos como EFR, ABR, respuestas neuronales y otros estados internos del modelo.

La aplicación de SDD + Antigravity debe ayudarnos a construir esos servicios de forma ordenada, no a reemplazar el análisis del equipo.

---

2. Idea central de SDD aplicada al proyecto

SDD significa **desarrollo guiado por especificaciones**.

La idea principal es:

> Antes de pedirle a Antigravity que genere código, primero debemos escribir con claridad qué debe hacer el servicio.

En el codelab, el flujo de SDD se basa en generar documentos antes de implementar: primero una especificación, después una planificación técnica, luego una lista de tareas y finalmente la implementación. Google explica que la especificación debe funcionar como fuente de verdad sobre la intención de la funcionalidad, para que el código generado pueda compararse contra lo pedido. (Google Codelabs)

Esto es importante porque trabajar directamente con IA sin especificaciones puede ser rápido al principio, pero riesgoso cuando el proyecto crece. El problema no es que la IA genere código, sino que genere código sin suficiente contexto, sin criterios claros de aceptación o sin respetar la arquitectura del sistema.

En nuestro proyecto, SDD se debería usar cada vez que se agregue o modifique un servicio relevante.

---

3. Qué entendemos por “servicio” en este proyecto

Para este apunte, un **servicio** es una parte funcional del sistema que cumple una responsabilidad concreta.

Ejemplos posibles de servicios:

- Servicio de carga de audiogramas.
- Servicio de validación de datos de entrada.
- Servicio de ejecución de simulaciones.
- Servicio de gestión de historial de simulaciones.
- Servicio de visualización de resultados.
- Servicio de autenticación de usuarios.
- Servicio de comparación entre simulaciones.

Cada servicio debe tener una responsabilidad clara. No conviene mezclar muchas tareas en un mismo servicio porque dificulta especificar, revisar y testear.

Ejemplo:

Servicio	Responsabilidad principal
`audiogram-upload-service`	Recibir y validar archivos de audiograma
`simulation-runner-service`	Ejecutar el modelo de simulación con parámetros válidos
`results-visualization-service`	Preparar los datos para mostrarlos en gráficos
`simulation-history-service`	Guardar y consultar simulaciones anteriores

La idea es que cada servicio tenga su propia carpeta de requerimientos para que Antigravity pueda trabajar con contexto claro.

---

4. Estructura recomendada de carpeta por servicio

Para aplicar SDD correctamente, cada servicio debería tener una carpeta propia dentro de una carpeta general de especificaciones.

Una estructura posible sería:

```
specs/
└── services/
    └── simulation-runner-service/
        ├── README.md
        ├── requirements.md
        ├── acceptance-criteria.md
        ├── inputs-outputs.md
        ├── business-rules.md
        ├── edge-cases.md
        ├── plan.md
        ├── tasks.md
        └── review-notes.md
```

Esta estructura está inspirada en el flujo del codelab, donde los artefactos de especificación se conservan como archivos dentro de `specs/<feature-branch>/`, versionados con Git, para que las decisiones no queden perdidas en una conversación con la IA. (Google Codelabs)

No todos los archivos tienen que ser extensos. Lo importante es que cada uno tenga una función clara.

---

5. Qué debe contener cada archivo

5.1 `README.md`

Sirve como resumen del servicio.

Debe responder:

- ¿Qué servicio es?
- ¿Qué problema resuelve?
- ¿Qué usuario o parte del sistema lo usa?
- ¿Con qué otros servicios se relaciona?

Ejemplo:

Simulation Runner Service

Este servicio se encarga de recibir datos auditivos ya validados y ejecutar una simulación basada en el modelo de Verhulst.

No valida archivos crudos ni genera gráficos finales. Su responsabilidad es preparar la ejecución, invocar el modelo correspondiente y devolver resultados procesables para otros servicios.

5.2 `requirements.md`

Es el archivo principal de requerimientos.

Debe describir **qué debe hacer el servicio**, sin entrar todavía en detalles técnicos excesivos.

Debe incluir:

- objetivo funcional;
- alcance;
- acciones principales;
- restricciones;
- dependencias;
- cosas que el servicio no debe hacer.

Ejemplo:

Requerimientos - Simulation Runner Service

Objetivo

Permitir la ejecución de simulaciones auditivas a partir de datos de entrada previamente validados.

Alcance

El servicio debe:
- recibir un conjunto de parámetros auditivos;
- recibir la referencia a un audiograma válido;
- ejecutar el modelo de simulación correspondiente;
- devolver un identificador de simulación;
- devolver el estado inicial de procesamiento;
- registrar errores si la simulación no puede iniciarse.

Fuera de alcance

El servicio no debe:
- validar el formato original del archivo cargado;

- interpretar clínicamente los resultados;
- generar gráficos interactivos;
- gestionar autenticación de usuarios.

Esto es importante porque Antigravity necesita saber no solo qué debe construir, sino también qué **no** debe construir.

5.3 `inputs-outputs.md`

Define entradas y salidas del servicio.

Debe especificar:

- datos que recibe;
- tipo de dato esperado;
- campos obligatorios;
- campos opcionales;
- formato de salida;
- errores posibles.

Ejemplo:

Entradas y salidas - Simulation Runner Service

Entradas

Campo	Tipo	Obligatorio	Descripción
user_id	string/int	Sí	Usuario que solicita la simulación
audiogram_id	string/int	Sí	Audiograma previamente cargado y validado
stimulus_type	string	Sí	Tipo de estímulo a usar en la simulación
frequency	number	Sí	Frecuencia del estímulo
intensity	number	Sí	Intensidad del estímulo
pathology_parameters	object	No	Parámetros patológicos configurables

Salidas esperadas

Si la simulación inicia correctamente:

```json
{
  "simulation_id": "sim_123",
  "status": "queued",
  "message": "Simulation started successfully"
}
```

Si ocurre un error:

```
{
  "error": "INVALID_PARAMETERS",
  "message": "Frequency value is outside the allowed range"
}
```

5.4 `business-rules.md`

Contiene reglas funcionales que el servicio debe respetar.

En este proyecto, estas reglas son especialmente importantes porque el sistema trabaja con datos científicos y simulaciones.

Ejemplo:

```md
# Reglas del servicio - Simulation Runner Service

- El servicio solo puede ejecutar simulaciones con audiogramas previamente validados.
- Toda simulación debe quedar asociada a un usuario.
- Toda simulación debe guardar fecha, hora, parámetros usados y estado.
- Si faltan parámetros obligatorios, la simulación no debe iniciarse.
- Si el modelo devuelve un error, debe registrarse el motivo.
- El servicio no debe modificar los datos originales del audiograma.
- El resultado debe poder ser consultado posteriormente desde el historial.
```

5.5 `acceptance-criteria.md`

Define cómo sabemos que el servicio está terminado.

Esto es clave para responder la pregunta del feedback:

> “¿Este documento tiene todo lo necesario para ya saber cómo usar eficientemente SDD con Antigravity en nuestro proyecto?”

Los criterios de aceptación deben ser concretos y verificables.

Ejemplo:

Criterios de aceptación - Simulation Runner Service

El servicio se considera terminado cuando:

- Permite iniciar una simulación con datos válidos.
- Rechaza solicitudes con parámetros incompletos.
- Rechaza solicitudes con audiogramas inexistentes o inválidos.
- Devuelve un identificador único de simulación.
- Guarda el estado inicial de la simulación.
- Permite consultar el estado de la simulación.
- Registra errores de ejecución.
- No bloquea la interfaz del usuario mientras se procesa.

Según el codelab, la especificación debe capturar requisitos funcionales y criterios de éxito antes de avanzar a la planificación. (Google Codelabs)

5.6 `edge-cases.md`

Lista casos problemáticos que Antigravity debe considerar.

Ejemplo:

Casos límite - Simulation Runner Service

- El usuario intenta ejecutar una simulación sin audiograma.
- El audiograma existe, pero está incompleto.
- Los parámetros ingresados están fuera del rango permitido.
- El modelo tarda demasiado en responder.
- El motor de simulación falla durante la ejecución.
- El usuario intenta iniciar varias simulaciones al mismo tiempo.
- Se pierde conexión durante el procesamiento.

Este archivo ayuda a que Antigravity no implemente solamente el “camino feliz”.

5.7 `plan.md`

Este archivo no se escribe completamente desde cero al principio. Se puede generar con Antigravity usando el comando de planificación.

Debe contener:

- enfoque técnico;
- archivos que se modificarán;
- servicios relacionados;
- decisiones técnicas;
- riesgos;
- dependencias.

En el codelab, la fase de planificación genera artefactos como `plan.md`, `data-model.md` y `research.md`, donde se define cómo construir técnicamente la funcionalidad. (Google Codelabs)

5.8 `tasks.md`

Debe contener tareas concretas y ordenadas.

Ejemplo:

```
# Tareas - Simulation Runner Service

- [ ] Crear estructura base del servicio.
- [ ] Definir modelo de datos de simulación.
- [ ] Crear endpoint para iniciar simulación.
- [ ] Validar existencia del audiograma.
- [ ] Validar parámetros obligatorios.
- [ ] Integrar llamada al motor de simulación.
- [ ] Guardar estado inicial de la simulación.
- [ ] Manejar errores del motor.
- [ ] Crear tests básicos del servicio.
- [ ] Documentar uso del servicio.
```

Antigravity puede generar una lista de tareas ordenada a partir del plan; el codelab muestra que estas tareas pueden incluir IDs, prioridades y rutas de archivos, organizadas por fases. (Google Codelabs)

5.9 `review-notes.md`

Sirve para registrar decisiones, dudas o cambios después de revisar lo generado por la IA.

Ejemplo:

```
# Notas de revisión - Simulation Runner Service

## Decisiones tomadas

- La validación de archivos queda fuera de este servicio.
- La simulación se ejecutará en segundo plano.
- El servicio devolverá un estado inicial `queued`.

## Pendientes
```

- Definir rangos válidos para cada parámetro fisiológico.
- Confirmar formato final de salida del modelo.

---

6. Flujo de trabajo recomendado con Antigravity

Para cada servicio, el flujo debería ser:

1. Crear carpeta del servicio
2. Escribir requerimientos
3. Definir entradas y salidas
4. Definir reglas y criterios de aceptación
5. Usar Antigravity para generar spec/plan/tasks
6. Revisar los documentos generados
7. Pedir implementación
8. Revisar código generado
9. Probar
10. Actualizar documentación si hubo cambios

En Antigravity, el codelab propone una secuencia basada en comandos como:

/speckit.specify
/speckit.clarify
/speckit.plan
/speckit.tasks
/speckit.analyze
/speckit.implement

Cada comando cumple una función distinta: especificar qué se construye, aclarar ambigüedades, planificar cómo se construye, generar tareas, analizar riesgos e implementar. (Google Codelabs)

Para nuestro proyecto, no siempre será necesario usar todos los pasos con la misma profundidad. En servicios simples, puede bastar con especificar, planificar e implementar. En servicios complejos, como ejecución de simulaciones o visualización de resultados científicos, conviene usar también aclaración y análisis.

---

7. Cómo escribir el pedido inicial para Antigravity

No conviene pedir:

Hacé el servicio de simulación.

Eso es demasiado ambiguo.

Conviene pedir:

Usando SDD, generá la especificación para el servicio simulation-runner-service.

El servicio debe recibir datos auditivos previamente validados, iniciar una simulación basada en el modelo correspondiente, guardar el estado inicial y devolver un identificador de simulación.

Usá como contexto los archivos:
- requirements.md
- inputs-outputs.md
- business-rules.md
- acceptance-criteria.md
- edge-cases.md

No implementes código todavía. Primero generá la especificación y marcá dudas si falta información.

Después de revisar la especificación, se puede avanzar con:

/speckit.plan

Planificá la implementación del servicio simulation-runner-service respetando la especificación aprobada.

Separá claramente:
- responsabilidades del servicio;
- archivos a crear o modificar;
- dependencias con otros servicios;
- modelo de datos necesario;
- riesgos técnicos;
- tareas previas a la implementación.

Y luego:

/speckit.tasks

Finalmente:

/speckit.implement

Pero solo después de revisar que el plan y las tareas tengan sentido.

---

8. Ejemplo completo aplicado a un servicio

Servicio elegido

simulation-runner-service

Responsabilidad

Ejecutar simulaciones auditivas a partir de datos previamente cargados y validados.

Carpeta

```
specs/
└── services/
    └── simulation-runner-service/
        ├── README.md
        ├── requirements.md
        ├── inputs-outputs.md
        ├── business-rules.md
        ├── acceptance-criteria.md
        ├── edge-cases.md
        ├── plan.md
        ├── tasks.md
        └── review-notes.md
```

`requirements.md`

```
# Requerimientos - Simulation Runner Service

## Objetivo

Permitir que un usuario ejecute una simulación auditiva a partir de datos previamente validados.

## Usuario principal

Profesional, investigador o integrante autorizado que necesita analizar una simulación auditiva.

## Alcance

El servicio debe:
- recibir la referencia a un audiograma válido;
- recibir parámetros de simulación;
```

- crear un registro de simulación;
- enviar la simulación al motor correspondiente;
- devolver un identificador único;
- permitir consultar el estado inicial.

Fuera de alcance

El servicio no debe:
- validar archivos originales;
- generar gráficos finales;
- interpretar clínicamente los resultados;
- administrar usuarios;
- modificar el audiograma original.

inputs-outputs.md

Entradas y salidas

Entrada mínima

- user_id
- audiogram_id
- simulation_parameters

Parámetros posibles

- tipo de estímulo
- frecuencia
- intensidad
- duración
- parámetros fisiológicos/patológicos configurables

Salida exitosa

- simulation_id
- status
- created_at
- message

Salida con error

- error_code
- message

acceptance-criteria.md

Criterios de aceptación

El servicio está terminado cuando:

- inicia simulaciones con datos válidos;
- rechaza datos incompletos;
- rechaza audiogramas inexistentes;
- guarda un registro de simulación;
- devuelve un identificador único;
- informa estado inicial;
- registra errores;
- no bloquea la interfaz mientras procesa;
- permite que otro servicio consulte el resultado luego.

---

9. Qué debe revisar el equipo antes de aceptar código generado

Antes de aceptar la implementación de Antigravity, se debe revisar:

- ¿El código respeta la responsabilidad del servicio?
- ¿Implementa exactamente lo especificado?
- ¿Agrega cosas no pedidas?
- ¿Respeta entradas y salidas?
- ¿Maneja errores?
- ¿Contempla casos límite?
- ¿Se conecta correctamente con otros servicios?
- ¿Es entendible para el equipo?
- ¿Se puede testear?
- ¿La documentación sigue coincidiendo con el código?

Si el código generado no coincide con la especificación, no se corrige “a mano” directamente como primera opción. Primero se revisa si la especificación estaba incompleta. Si estaba incompleta, se actualiza la especificación y luego se vuelve a pedir ajuste.

---

10. Cuándo usar cada etapa

Etapa	Cuándo usarla	Resultado esperado
`specify`	Cuando se define un servicio o funcionalidad nueva	Especificación clara
`clarify`	Cuando hay dudas, supuestos o ambigüedades	Preguntas y respuestas incorporadas

`plan`	Antes de escribir código	Plan técnico
`tasks`	Antes de implementar	Lista de tareas
`analyze`	Si el servicio es complejo o toca varias partes	Detección de riesgos
`implement`	Cuando spec, plan y tasks están revisados	Código generado

---

11. Reglas internas para usar Antigravity en el proyecto

Para que Antigravity sea útil y no genere desorden, el equipo debería seguir estas reglas:

1. No pedir implementación sin requerimientos escritos.
2. No mezclar varios servicios en una misma especificación.
3. No aceptar código sin revisar plan y tareas.
4. No dejar decisiones importantes solo en el chat.
5. Guardar cada especificación en su carpeta correspondiente.
6. Definir siempre entradas, salidas, errores y criterios de aceptación.
7. Actualizar la documentación si cambia el comportamiento del servicio.
8. Usar `clarify` cuando Antigravity detecte dudas reales.
9. Usar `analyze` en servicios complejos.
10. Mantener el foco en servicios concretos, no en “todo el sistema” al mismo tiempo.

---

12. Checklist final para saber si un servicio está listo para Antigravity

Antes de pedir implementación, el equipo debería poder responder “sí” a estas preguntas:

[ ] ¿Está claro qué hace el servicio?
[ ] ¿Está claro qué NO hace el servicio?
[ ] ¿Están definidas las entradas?
[ ] ¿Están definidas las salidas?
[ ] ¿Están definidos los errores posibles?
[ ] ¿Hay criterios de aceptación verificables?
[ ] ¿Hay casos límite documentados?
[ ] ¿Se sabe con qué otros servicios se conecta?
[ ] ¿Se puede probar el resultado?
[ ] ¿La IA tiene suficiente contexto para no inventar arquitectura?

Si alguna respuesta es “no”, todavía falta completar la especificación antes de implementar.

---

13. Conclusión operativa

Para usar eficientemente SDD con Antigravity en el Laboratorio Virtual Auditivo, el equipo debe trabajar por **servicios**, no por pedidos generales.

Cada servicio debe tener una carpeta de requerimientos con:

- qué hace;
- qué no hace;
- entradas;
- salidas;
- reglas;
- criterios de aceptación;
- casos límite;
- plan;
- tareas;
- notas de revisión.

Antigravity debe usarse después de esa definición, no antes. Su función es acelerar la generación de especificaciones, planes, tareas e implementación, pero el equipo sigue siendo responsable de revisar que el resultado respete el objetivo del proyecto.

La pregunta de cierre para validar el documento sería:

> ¿Este documento tiene todo lo necesario para saber cómo usar eficientemente SDD con Antigravity en nuestro proyecto?

Con esta estructura, la respuesta debería ser **sí**, porque el equipo ya tendría una forma concreta de organizar requerimientos por servicio, pedirle trabajo a Antigravity, revisar lo generado y avanzar sin depender de instrucciones vagas o conversaciones sueltas.